

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

Name: Prabhavathi R **Register No:** 192321101

Course: Computer Vision **Course Code:** ITA05 **CO5**

Assessment Tool Weightage: 40%

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. The answers below follow the 15 problem statements in the supplied Assessment Tool 1 and address their stated constraints through complete OpenCV-oriented pipelines.

1. Real-Time Face Recognition System (Edge Deployment): Design a complete OpenCV solution satisfying real-time, lighting, resource, and predefined-dataset constraints.

Use a lightweight webcam/video capture with cv2.VideoCapture and resize frames before processing. Convert to grayscale or a normalized color representation and apply illumination correction such as histogram equalization/CLAHE. Detect faces using a lightweight Haar/LBP cascade or an efficient OpenCV DNN face detector. For each detected face, crop the ROI, resize it to the recognizer input size and normalize it. Use a pretrained recognizer/model trained on the predefined dataset, such as an LBPH recognizer for a lightweight classical solution. Draw the bounding box and identity label on the original frame. To satisfy the constraints, keep the processing resolution modest, reuse loaded models, avoid unnecessary copies, process only detected ROIs for recognition, and use efficient frame loops. CLAHE improves robustness to lighting changes, while a lightweight detector/recognizer reduces CPU and memory use. A confidence threshold should reject uncertain matches instead of producing false identities.

2. Automated Video Surveillance with Alert System: Design an OpenCV system that detects motion/unusual activity, produces real-time alerts, accepts CCTV feeds, and minimizes false positives.

Read the CCTV stream with cv2.VideoCapture. Convert frames to grayscale and blur them to suppress sensor noise. Estimate the background using a background-subtraction method such as MOG2 or KNN. Apply thresholding and morphological opening/closing to remove small noise and connect meaningful regions. Find contours and reject regions below a minimum area. Track valid moving objects across consecutive frames and require motion to persist for several frames before raising an alert. Draw regions, IDs, and alert status on the output frame. Dynamic backgrounds are handled better by adaptive background models than by simple fixed frame differencing. Blur, morphology, minimum-area filtering, persistence checks, and tracking reduce false positives from noise, shadows, leaves, or brief illumination changes. Processing at a suitable resolution keeps the system near real time.

3. OCR System for Low-Quality Documents: Design an OpenCV pipeline for noisy, low-resolution scanned documents with varying fonts and illumination and efficient batch processing.

For every input image, load it with cv2.imread, resize if necessary, and convert it to grayscale. Improve illumination and contrast using CLAHE or histogram equalization. Apply denoising such as Gaussian or median filtering. Use adaptive thresholding or Otsu thresholding to obtain a clear binary text image. Morphological opening can remove isolated noise, while closing can reconnect broken character strokes. Optionally deskew the document before segmentation. Detect text regions/lines and pass the cleaned regions to an OCR engine. Store the recognized text

and process images one at a time to limit memory use. The sequence separates enhancement, segmentation, and recognition. Adaptive thresholding is useful when illumination varies across the page, while denoising and morphology improve character structure. Sequential batch processing avoids keeping all high-resolution images in memory.

4. Facial Expression Recognition for Mobile Application: Design an efficient OpenCV solution for happy, sad, and neutral expressions under mobile CPU, real-time, head-movement, and lighting constraints.

Capture frames from the mobile camera and reduce their resolution. Convert to grayscale and apply light contrast normalization. Detect the face with a lightweight OpenCV detector. Track the detected face between detections so that full detection is not required on every frame. Align or normalize the face ROI and extract compact facial features, for example local texture/landmark-based features. Classify the features into happy, sad, or neutral using a lightweight trained classifier/model. Smooth predictions over several frames to prevent flickering labels and display the result.

Efficiency comes from downsampling, ROI-only processing, detector reuse/tracking, and a compact classifier. Illumination normalization improves robustness, while tracking and temporal smoothing handle slight head movement and frame-to-frame variation.

5. Smart Traffic Monitoring System: Design an OpenCV system to detect vehicles, track movement, count vehicles, display statistics, and operate continuously with minimal delay.

Read the live stream using `cv2.VideoCapture`. Resize frames to a processing resolution and define a road ROI. Detect vehicles using a suitable OpenCV DNN detector or background subtraction for a fixed camera. For each detection, create a bounding box and centroid. Associate detections across frames using centroid tracking or a lightweight tracker. Define a virtual counting line/zone and increment the vehicle count only when a tracked ID crosses it in the intended direction. Draw boxes, IDs, the counting line, and totals using `cv2.rectangle`, `cv2.putText`, and `cv2.line`. Tracking IDs prevent the same vehicle from being counted repeatedly. ROI restriction reduces unnecessary computation and background detections. A compact detector, resized frames, and efficient tracking maintain low latency for continuous operation.

6. Real-Time Object Detection on Mobile Device: Design an OpenCV pipeline from camera acquisition to efficient detection and display under CPU and memory constraints.

Open the camera and continuously capture frames. Resize each frame to the detector's required input size and normalize it. Run a lightweight pretrained object-detection model through OpenCV DNN. Apply confidence filtering and non-maximum suppression to remove weak and duplicate detections. Draw bounding boxes, class labels, and confidence scores, then display the result. Release the camera and destroy windows when the loop ends. The main optimization is to use a compact model and small input resolution. Processing only one current frame at a time limits memory usage, while NMS and confidence thresholds improve output quality. If needed, detection can be performed on every second frame while tracking is used between detections.

7. Smart Attendance System using Face Recognition: Design an OpenCV pipeline that captures faces, recognizes registered users, marks attendance, and avoids false recognition.

Capture frames from a webcam and detect faces. For each face, preprocess and normalize the ROI, then compare it with the registered face dataset using an appropriate OpenCV recognizer/model. Accept a match only when the

recognition confidence passes a strict threshold. Require the same identity to be observed consistently across multiple frames before marking attendance. Store the name, date, and time in an attendance file/database and prevent duplicate entries for the same session. Draw the identity and status on the frame. Strict confidence thresholds reduce false recognition, while multi-frame confirmation makes the decision more stable. A duplicate check ensures one attendance entry per person per session.

8. Motion Detection for Home Security: Design a pipeline that detects significant movement, triggers alerts, and reduces false alarms from noise and background variation.

Read the camera feed and initialize an adaptive background model. Convert frames to grayscale, blur them, and perform background subtraction. Threshold the foreground mask and use morphological opening/closing to remove noise. Find contours and ignore very small regions. Define a minimum motion area and require motion to remain present for a short number of consecutive frames before triggering an alert. Optionally restrict detection to important zones such as doors/windows. Draw the detected region and alert status. Adaptive background modeling handles gradual scene changes. Blur, morphology, area filtering, ROI restriction, and persistence checks suppress false alarms caused by sensor noise, shadows, or tiny background movements.

9. Document Scanner Application: Design an OpenCV pipeline that captures documents, improves readability, and produces clean scanned images.

Capture an image from the camera and resize it for processing. Convert it to grayscale and apply Gaussian/median filtering. Use Canny edge detection to locate strong document boundaries. Find contours and select a large four-corner contour representing the page. Apply a perspective transform using the four corners to obtain a top-down view. Convert the warped image to grayscale and apply adaptive thresholding or contrast enhancement. Crop/clean the result and save or display the final scan. Perspective correction removes skew caused by camera angle. Edge detection and contour selection locate the page, while thresholding and contrast enhancement improve readability.

10. Vehicle Detection in Parking System: Design a pipeline to detect entering/exiting cars, track movement, update available slots, and avoid counting errors.

Read the parking-camera stream and define the parking/entry ROI. Detect vehicles using an OpenCV DNN detector or background subtraction for a fixed view. Generate bounding boxes and centroids and assign tracking IDs. Define entry and exit counting lines. Update the vehicle count only when a tracked ID crosses a line in a valid direction, and maintain a set of already-counted IDs. Available slots = total slots minus currently occupied slots. Draw vehicle boxes, IDs, lines, and the slot count. Tracking IDs and line-crossing logic prevent double counting. Restricting detection to the relevant ROI improves speed, while direction checks distinguish entry from exit.

11. Gesture Recognition System: Design an OpenCV webcam pipeline that identifies simple hand gestures and responds in real time.

Capture webcam frames and convert them to a suitable color space such as HSV. Segment the hand region using skin/color or background-based thresholding, then denoise the mask with morphology. Find contours and select the hand contour by area. Extract compact shape features such as contour area, perimeter, convex hull, convexity defects, or fingertip count. Classify simple gestures using rule-based thresholds or a lightweight classifier. Map each recognized gesture to an application action and display the result. Contour-based features are computationally inexpensive and suitable for real-time operation. Morphological filtering improves segmentation, while area and shape checks reduce false detections.

12. Video Frame Quality Enhancement: Design a real-time pipeline to reduce noise and enhance contrast while preserving performance.

Capture each video frame and resize it if necessary. Apply a lightweight denoising filter such as Gaussian or median filtering. Convert to an appropriate color representation and enhance luminance/contrast using histogram equalization or CLAHE. Optionally apply mild sharpening using an unsharp-mask style operation. Display the enhanced frame and continue until the user exits. The denoising stage reduces sensor noise, while contrast enhancement improves visibility in dark or unevenly illuminated scenes. Processing at a controlled resolution and using lightweight filters keeps the pipeline suitable for real-time video.

13. Multi-Face Tracking System: Design an OpenCV pipeline that detects multiple faces, tracks them across frames, and maintains identity.

Capture frames and detect multiple faces using a Haar/LBP detector or OpenCV DNN face detector. Assign a unique tracking ID to each detected face. In subsequent frames, update the locations using a tracker or associate new detections with existing tracks based on centroid/IOU proximity. Run recognition only when required to refresh identity labels. Remove tracks that disappear for a defined number of frames. Draw each face box, ID, and identity. Tracking reduces the need for expensive recognition on every frame. Unique IDs and temporal association maintain identity, while track timeouts prevent stale identities from remaining on screen.

14. OCR for License Plate Recognition: Design an OpenCV pipeline to detect license plates, extract text, and work under varying lighting.

Read vehicle images/video and optionally restrict processing to a vehicle ROI. Convert to grayscale and normalize illumination using CLAHE. Apply edge detection or a suitable threshold to identify plate-like rectangular regions. Use contours and geometric constraints such as aspect ratio and area to select candidate plates. Crop and rectify the selected plate, resize it, denoise it, and apply adaptive thresholding. Segment characters if required and pass the cleaned plate image to an OCR engine. Draw the plate box and recognized text. Illumination normalization and adaptive thresholding handle uneven lighting. Geometric filtering removes many non-plate regions, while perspective correction and resizing make characters easier for OCR.

15. Real-Time Edge Detection System: Design an efficient OpenCV video pipeline that captures video, detects edges, and displays results with minimal delay.

Capture frames from the camera and resize them to a suitable processing resolution. Convert each frame to grayscale and apply Gaussian blur to suppress noise. Use Canny edge detection with suitable low/high thresholds. Optionally perform a small morphological operation if edge continuity is required. Display the edge image or overlay edges on the original frame. Stop when the exit key is pressed and release resources. Gaussian blur reduces false edges caused by noise, while Canny provides clear gradient-based edge detection. Resizing, lightweight preprocessing, and avoiding unnecessary copies reduce latency and support real-time performance.

Student Declaration

I, Prabhavathi R(192321101), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the student: _____Prabhavathi R_____